

DuckDB from the Command Line

A Second Way to Talk to Your Database — Windows

Table of Contents

1. Why Learn the Command Line?
2. Two Different "DuckDBs"
3. What You'll Need
4. Installing the DuckDB CLI on Windows
5. Verifying the Install
6. Starting DuckDB
7. The DuckDB Prompt
8. Dot-Commands — Your Toolbox
9. Changing How Results Look
10. Querying a CSV — No Loading Step
11. One-Off Queries and Script Files

Why Learn the Command Line?

So far, you've used DuckDB **inside Marimo notebooks**. That's great for building and sharing analysis.

The **command line** is different — it's a fast, no-frills way to:

- Quickly check a table without opening a notebook
- Run a `.sql` file as part of a script or pipeline
- Peek inside a `.csv` file in seconds
- Practice SQL the way many working analysts actually do

👉 Notebooks and the command line are **two tools for the same job**.

Knowing both makes you more flexible.

Two Different "DuckDBs"

You already have the **DuckDB Python package** installed (`pip install duckdb`). That's what powers `import duckdb` in Marimo.

Today we install a **second, separate thing**: the **DuckDB CLI** — a standalone program called `duckdb.exe` that you run directly in a terminal, with no Python involved at all.

	Python package	DuckDB CLI
How you use it	<code>import duckdb</code> in a script/notebook	Type <code>duckdb</code> in a terminal
Needs Python?	Yes	No
Best for	Notebooks, analysis with pandas	Quick checks, scripts, learning SQL

What You'll Need

- A terminal — either works, but they behave a little differently:
 - **Command Prompt** (press `Win`, type `cmd`, press Enter)
 - **PowerShell** (press `Win`, type `powershell`, press Enter)
- 5–10 minutes to install the DuckDB CLI (next slides)
- The sample file `data/students.csv` in this same tutorial folder
- **Windows 10 or 11**

That's it — no server, no account, no configuration file.

Installing the DuckDB CLI — Option A: `winget` (recommended)

`winget` is Microsoft's own package manager, built into Windows 10 (1709+) and Windows 11. Check you have it:

```
winget --version
```

If that prints a version number, install DuckDB with one command (works the same in Command Prompt or PowerShell):

```
winget install DuckDB.cli
```

Watch for a line ending in `Successfully installed`. If `winget` asks you to accept a source agreement the first time you ever use it, type `Y` and press Enter.

Installing the DuckDB CLI — Option B: Manual Download

No `winget`, or it's blocked by school-managed device policy? Get the binary directly:

1. Go to **duckdb.org/install** and choose **Windows + CLI**
2. Download the `.zip` (`duckdb_cli-windows-amd64.zip` for almost everyone — Windows on Arm is rare and out of scope for this course)
3. Right-click the downloaded `.zip` in **Downloads** and choose **Extract All...** — you get a single file named `duckdb.exe`
4. Move `duckdb.exe` somewhere permanent, e.g. create the folder `C:\duckdb\` and move it there
5. Add that folder to your **PATH** (next slide) so Windows can find `duckdb` from any folder

Adding `duckdb.exe` to Your PATH

Only needed for the manual-download route — `winget` does this step for you automatically.

1. Press `Win`, type `environment variables`, open **"Edit the system environment variables"**
2. Click the **"Environment Variables..."** button
3. Under **"User variables"**, select `Path`, click **"Edit..."**
4. Click **"New"**, type `C:\duckdb`, click **OK** on all three open windows
5. **Close every open Command Prompt / PowerShell window** — the PATH change only applies to *new* terminal windows

"Windows protected your PC" — SmartScreen Warning

If you try to run the manually downloaded `duckdb.exe` and see a blue **"Windows protected your PC"** screen, that's **SmartScreen** — the file isn't digitally signed the way a Microsoft Store app is.

Click **"More info"**, then **"Run anyway"**. This is a one-time prompt per file. `winget install DuckDB.cli` never triggers this warning, which is the main reason this course recommends it over the manual download.

Verifying the Install

Open a brand-new Command Prompt or PowerShell window (PATH changes never apply to windows already open), then run:

```
duckdb --version
```

You should see something like:

```
v1.1.3 19864453f7
```

Confirm **where** it's running from:

```
where duckdb
```

(PowerShell also understands `where.exe duckdb`, or the PowerShell-native `Get-Command duckdb`.)

Starting DuckDB — In-Memory

Just type `duckdb` with nothing after it:

```
duckdb
```

```
v1.1.3 19864453f7
Enter ".help" for usage hints.
Connected to a transient in-memory database.
Use ".open FILENAME" to reopen on a persistent database.
D
```

The `D` is your **prompt** — DuckDB is waiting for a command.

⚠ "In-memory" means: when you close this window, **everything you created is gone**. Good for quick experiments.

Starting DuckDB — Persistent File

To **save** your work to disk, give `duckdb` a filename:

```
duckdb my_class.db
```

- If `my_class.db` doesn't exist yet, DuckDB **creates** it.
- If it already exists, DuckDB **reopens** it — your tables are still there.

👉 This is exactly like the difference between

```
duckdb.connect(':memory:') and duckdb.connect('my_class.db')
```

in your Marimo notebooks.

An absolute Windows path also works, but **use forward slashes**, not the backslashes File Explorer shows you (see "Common Mistakes" ahead):

```
duckdb C:/Users/maria/Desktop/OMIS-105-LABS/my_class.db
```

The DuckDB Prompt

Once you see the `D` prompt, you can type SQL directly:

```
D SELECT 6 * 7 AS answer;
```

answer
int32
42

Press **Enter** after the `;` to run the statement.

Don't Forget the Semicolon

If you forget the `;`, DuckDB just waits for more input:

```
D SELECT 6 * 7 AS answer
...>
```

Type the `;` on the next line and press Enter to finish it.

⚠ This is the #1 "why isn't anything happening?" moment for beginners. When the prompt changes to `...>`, DuckDB is still listening — it just needs your semicolon.

Dot-Commands — Your Toolbox

Commands that start with a **dot (.)** are not SQL — they're DuckDB CLI shortcuts. No semicolon needed.

```
.help      show all dot-commands
.tables    list tables in the current database
.schema    show CREATE TABLE statements
.mode      change how results are displayed
.headers   show/hide column names
.timer     show how long each query took
.quit      exit the CLI
```

👉 Think of dot-commands as **settings for your session**, and SQL as **questions for your data**.

.tables and .schema

Create a table, add some rows, then inspect it:

```
D CREATE TABLE students (  
  id INTEGER,  
  name VARCHAR,  
  gpa DOUBLE);  
  
D INSERT INTO students VALUES  
  (1, 'Alice', 3.8), (2, 'Bob', 3.5),  
  (3, 'Charlie', 3.9), (4, 'Diana', 3.2),  
  (5, 'Ethan', 2.9), (6, 'Fiona', 3.7),  
  (7, 'George', 3.1), (8, 'Hana', 3.95);  
  
D .tables  
students  
  
D .schema students  
CREATE TABLE students(id INTEGER, name VARCHAR, gpa DOUBLE);
```

`.tables` answers *"what tables exist?"*

`.schema` answers *"what do they look like?"*

.mode — Changing How Results Look

The default look (`box` , shown so far) is nice to read, but not easy to paste elsewhere. Switch styles with `.mode` :

```
D .mode csv
D SELECT * FROM students LIMIT 2;
id,name,gpa
1,Alice,3.8
2,Bob,3.5
```

Other useful modes: `.mode markdown` (great for pasting into a README), `.mode json`, `.mode line`. Switch back anytime with `.mode box`.

.headers and .timer

```
D .headers off
D SELECT name FROM students LIMIT 1;
Alice
```

```
D .timer on
D SELECT COUNT(*) FROM students;
```

count_star()
8

```
Run Time (s): real 0.001 user 0.000539 sys 0.000315
```

`.timer on` is handy once your tables get big and you want to know if a query is fast or slow.

Querying a CSV — No Loading Step

This is DuckDB's superpower, and it works the same on the command line as it does in a notebook. No `CREATE TABLE`, no import wizard — the file **is** the table.

First, get into this tutorial's folder. **Easiest way** — open the folder in File Explorer, click the address bar, type `cmd`, press Enter (opens Command Prompt already pointed there). **Or manually:**

```
cd %USERPROFILE%\Desktop\OMIS-105\tutorials\DuckDB_from_Command_Line
```

(Adjust the path if you cloned or unzipped the course repo somewhere else. Confirm with plain `cd` and `dir data.`)

Querying a CSV — Running the Query

```
D SELECT major, ROUND(AVG(gpa), 2) AS avg_gpa
FROM read_csv('data/students.csv')
GROUP BY major
ORDER BY avg_gpa DESC;
```

major	avg_gpa
Information Systems	3.83
Accounting	3.50
Marketing	3.35
Finance	3.35

'data/students.csv' uses a **forward slash** and is a **relative path** — it only resolves correctly if your terminal is standing in this tutorial's folder (see the previous slide). DuckDB accepts forward slashes on Windows even though File Explorer shows backslashes.

One-Off Queries with `-c`

Sometimes you don't want to open the interactive prompt at all — you just want **one quick answer**. Run it straight from your terminal (not from inside `duckdb`) using `-c`.

Command Prompt:

```
duckdb -c "SELECT COUNT(*) FROM read_csv('data/students.csv')"
```

PowerShell (same syntax works too):

```
duckdb -c "SELECT COUNT(*) FROM read_csv('data/students.csv')"
```

count_star()
8

Quoting on Windows — Command Prompt vs. PowerShell

Both shells accept the pattern above (**double quotes around the whole SQL statement, single quotes inside for string literals** like `'data/students.csv'` or `WHERE major = 'Finance'`), but they diverge if you go further:

- **Command Prompt** has no concept of single-quoted strings at all — `'...'` is passed through literally, so keep using it *only* inside the double-quoted SQL, never as the outer quote.
- **PowerShell** treats single quotes (`'...'`) as **literal strings** (no variable expansion) and double quotes as **interpolating strings** (a `$name` inside would be substituted). This rarely bites you with `-c "SELECT ..."`, but if a query ever needs a literal `$`,

Running a Script File

1. Create the file. Open any plain-text editor (VS Code, Notepad — not Word), type the SQL below, and save it as `report.sql` in this same folder:

```
SELECT major, COUNT(*) AS n
FROM read_csv('data/students.csv')
GROUP BY major;
```

2. Run it. Back in your terminal (not inside the `duckdb` prompt):

```
duckdb -c ".read report.sql"
```

👉 `report.sql` is just text — DuckDB reads it and runs each statement in order. Save once, rerun anytime: that's **repeatable** SQL.

Leaving DuckDB

Any of these will exit the interactive prompt:

```
D .quit
```

```
D .exit
```

Or press **Ctrl+Z** then **Enter** — Windows' "end of input" shortcut (different from Ctrl+D on Mac/Linux).

⚠ If you used an **in-memory** session, everything you built disappears the moment you exit. Use a file (`duckdb my_class.db`) if you want it to survive.

Common Mistakes

- **Forgetting the `;`** → prompt hangs at `...>`. Type `;` + Enter.
- **Typing a dot-command with a `;`** → dot-commands don't need one (`.tables;` will error).
- **Pasting a raw backslash path** (`C:\Users\maria\data.csv`) into a `read_csv(...)` call → use forward slashes instead: `C:/Users/maria/data.csv`. This is the same backslash issue that trips up Python paths in Marimo notebooks — DuckDB and Windows both quietly accept `/`, so standardize on it.

Common Mistakes (continued)

- **Using an in-memory session, then closing the window** → your table is gone. Use a `.db` file if you need it saved.
- **Running `duckdb` from the wrong folder** → `read_csv('data/students.csv')` only works if your terminal is in this tutorial's folder. Check with plain `cd .`
- **Installed with `winget`, but `'duckdb'` is not recognized** (Command Prompt) **or `The term 'duckdb' is not recognized as the name of a cmdlet...`** (PowerShell) → same cause either way: you're still in the terminal window that was open *before* the install finished. Open a brand-new window.

Troubleshooting

'duckdb' is not recognized as an internal or external command

Seeing this in **PowerShell** instead? Same problem, different wording: **The term 'duckdb' is not recognized as the name of a cmdlet, function, script file, or operable program.** The fix below is identical for both shells.

1. Open a **brand-new** Command Prompt or PowerShell window — PATH changes never reach windows that were already open.
2. If you installed manually, confirm the folder you added really contains **duckdb.exe**: **dir C:\duckdb**.
3. Re-check the PATH entry: **Win** → "environment variables" → "Edit the system environment variables" → "Environment

Troubleshooting (continued)

SmartScreen keeps blocking `duckdb.exe`

See the SmartScreen slide earlier — click **"More info"** → **"Run anyway"**. If your device is school- or company-managed and that button is missing, switch to `winget install DuckDB.cli`, which installs from Microsoft's own package source and doesn't trigger SmartScreen.

My CSV query says "No files found that match the pattern"

You're not standing in this tutorial's folder, or you pasted a raw backslash path. Run plain `cd` to check your current folder, and make sure any path inside `read_csv(...)` uses forward slashes.

Troubleshooting (continued)

OneDrive is messing with my path or file

If your Desktop or Documents folder is backed up by OneDrive (common on school-managed laptops), your real path may include `OneDrive` in it, e.g. `C:/Users/maria/OneDrive/Desktop/...`. Use File Explorer's **"Copy as path"** on the actual folder rather than typing a path from memory. OneDrive can also **lock** a `.db` file while syncing it, which shows up as a confusing "unable to open database" error even though the path is correct — if that keeps happening, work from a non-synced folder such as `C:\0MIS105-LABS\` instead.

`winget` itself isn't found, or refuses to run

`winget` ships with the **App Installer** package from the Microsoft

Store. On an older or locked down Windows 10 install it may be

More "What If...?" Scenarios

- **Results table shows garbled characters like `Γöî` instead of clean lines?** → legacy Command Prompt's default codepage (437) doesn't support the Unicode box-drawing characters DuckDB draws by default. Don't reach for `chcp 65001` — it's been reported to make DuckDB CLI queries hang instead of fixing the display. Use `.mode table` instead: it prints a plain `+----+` table that doesn't depend on the codepage at all.

- **Random/garbled characters appear and don't go away, specifically when running as Administrator?** → a known, unresolved DuckDB CLI bug on Windows 11 (Command Prompt and PowerShell both affected). There's no fix yet — just close that window and open a normal (non-Administrator) one. This course's steps never need Administrator anyway.
- **Opening a `.db` file says `I/O Error: Could not set lock on file...`?** → another terminal window — or a session you forgot to `.quit` — already has that file open; the message even names the PID holding it. Close that window, or run `.quit` there, then retry.

- `winget install DuckDB.cli` finishes but `duckdb.exe` is just... **gone from Downloads?** → different from SmartScreen — some school-managed endpoint tools quarantine or delete unsigned `.exe` files after download. Prefer `winget`, which installs from a source most device policies already trust, or ask IT for an exception.
- **Folder name has spaces, like `C:\Users\maria\OMIS 105 Labs` ?** → only the **shell** cares — quote it:
`cd "C:\Users\maria\OMIS 105 Labs"`. A path written as a SQL string, like `'data/students.csv'`, needs no extra escaping.

- **"Edit the system environment variables" greyed out or blocked by school policy?** → you don't need admin rights for your **own** PATH — these steps already edit the **User variables** Path, not the System one. If even that's locked down, skip PATH for now: run `set PATH=%PATH%;C:\duckdb` (Command Prompt) or `$env:PATH += ";C:\duckdb"` (PowerShell) once per session, or just type `C:\duckdb\duckdb.exe` in full each time.

☞ None of these are common — but now you'll recognize them instantly instead of guessing.

Cheat Sheet

Command	What it does
<code>winget install DuckDB.cli</code>	install the DuckDB CLI (recommended)
<code>duckdb --version</code>	confirm the install worked
<code>where duckdb</code>	show which copy of <code>duckdb</code> will run
<code>duckdb</code>	start in-memory session
<code>duckdb file.db</code>	start/reopen a persistent database
<code>duckdb -c "SQL"</code>	run one query from your terminal, then exit
<code>duckdb -c ".read file.sql"</code>	run a whole script file
<code>.tables</code>	list tables
<code>.schema [table]</code>	show table structure
<code>.mode box csv markdown json</code>	change output style

Practice Exercise

In your terminal, `cd` into this tutorial's folder, then:

1. Confirm the install: `duckdb --version` and `where duckdb`
2. Start DuckDB: `duckdb`
3. Run: `SELECT * FROM read_csv('data/students.csv');`
4. Find the average GPA **per major**, highest first
5. Switch to `.mode markdown` and rerun your query
6. Save your query in a file called `my_query.sql`,
then run it with `duckdb -c ".read my_query.sql"`
7. Exit with `.quit` or **Ctrl+Z, Enter**

👉 If you can do all seven steps, you can use DuckDB from the command line on Windows with confidence.

You're Ready

Notebooks for building analysis.

The command line for quick answers and repeatable scripts.

Resources

duckdb.org/docs/api/cli — Official CLI documentation

duckdb.org/install — Install DuckDB for Windows (winget or manual)

OMIS 105 — Introduction to Database Management Systems — Fall 2026